Title:

Design and Implementation of a Moore Finite State Machine for 101 Sequence Detection using Verilog HDL

Aim:

To design and implement a Moore Finite State Machine (FSM) using Verilog HDL to detect the input sequence 101 and verify its operation through simulation using ModelSim.

Software required:

Verilog HDL
ModelSim simulator
Computer or laptop

Theory:

A Finite State Machine (FSM) is a sequential circuit that changes its state according to the present state and the input.
There are two main types of FSM:
Moore Machine: Output depends only on the present state.
Mealy Machine: Output depends on the present state and input.
In this project, a Moore FSM is designed to detect the binary sequence 101.
Input: x
Clock: clk
Reset: reset
Output: z
When the input sequence 101 is detected, the output z becomes 1. Otherwise, the output remains 0, except for the state timing associated with the detection. Because this is a Moore machine, the output changes according to the state, not directly according to the input.
We will design it to support overlapping sequences. For example, the input 10101 contains two occurrences of 101.

State Diagram:

The FSM contains four states:
S0: No matching bits detected.
S1: The first bit 1 is detected.
S2: The sequence 10 is detected.
S3: The complete sequence 101 is detected.

State Transition Diagram:

S0 / 0
1 ↓
S1 / 0
0 ↓
S2 / 0
1 ↓

S3 / 1
Sequence detected: 101
This is a simplified path. The complete transition rules are provided in the table below.

State Encoding Table:

State	Binary Encoding	Output
S0	00	0
S1	01	0
S2	10	0
S3	11	1

State Transition Table:

Present State	Input x	Next State	Output z
S0	0	S0	0
S0	1	S1	0
S1	0	S2	0
S1	1	S1	0
S2	0	S0	0
S2	1	S3	0
S3	0	S2	1
S3	1	S1	1

The output depends only on the present state, which makes this a Moore FSM.

Verilog Design Code:
Create a file named moore_101_detector.v and enter the following code.

```
module moore_101_detector (
   input wire clk,
   input wire reset,
   input wire x,
   output wire z
);

   reg [1:0] state;

   localparam S0 = 2'b00,
              S1 = 2'b01,
              S2 = 2'b10,
              S3 = 2'b11;

   always @(posedge clk or posedge reset) begin
      if (reset)
         state <= S0;
```

```
        else begin
            case (state)
                S0: state <= x ? S1 : S0;
                S1: state <= x ? S1 : S2;
                S2: state <= x ? S3 : S0;
                S3: state <= x ? S1 : S2;
                default: state <= S0;
            endcase
        end
    end

    assign z = (state == S3);

endmodule
```

Code Explanation:
clk: Clock signal used to synchronize state changes.
reset: Resets the FSM to the initial state S0.
x: Serial binary input.
state: Stores the current state.
case: Determines the next state based on the current state and input.
z: Becomes 1 when the FSM is in state S3.

Testbench Code:
Create another file named tb_moore_101_detector.v.

```
`timescale 1ns/1ps

module tb_moore_101_detector;

    reg clk;
    reg reset;
    reg x;
    wire z;

    moore_101_detector uut (
        .clk(clk),
        .reset(reset),
        .x(x),
        .z(z)
    );

    always #5 clk = ~clk;
```

```
    initial begin
        clk = 0;
        reset = 1;
        x = 0;

        #12;
        reset = 0;

        @(negedge clk) x = 1;
        @(negedge clk) x = 0;
        @(negedge clk) x = 1;

        // Test overlapping detection
        @(negedge clk) x = 0;
        @(negedge clk) x = 1;

        @(negedge clk) x = 0;
        @(negedge clk) x = 0;

        #10;
        $finish;
    end

    initial begin
        $monitor("Time=%0t reset=%b x=%b z=%b",
                $time, reset, x, z);
    end

endmodule
```

ModelSim Simulation Procedure:

Open ModelSim.
Create a new project named Moore_101.
Add moore_101_detector.v to the project.
Add tb_moore_101_detector.v to the project.
Compile both Verilog files.
Select tb_moore_101_detector as the simulation top module.
Add the signals clk, reset, x, and z to the Wave window.
Run the simulation.
Observe the output waveform and verify the detection of 101.

Expected Output:

For the input sequence 101, the FSM reaches state S3 after receiving all three bits. The output becomes 1 while the FSM is in S3.

Input	received State	Output
1	S1	0
10	S2	0
101	S3	1

Capture the actual waveform from ModelSim and include it in your assignment as the simulation result.

Result:

The Moore FSM-based 101 sequence detector was designed using Verilog HDL. The design can be simulated using ModelSim to verify the output for the specified input sequence

Conclusion:

Thus, the Moore Finite State Machine for detecting the binary sequence 101 was designed using Verilog HDL. The design uses four states and supports overlapping sequence detection. The functionality can be verified through ModelSim simulation.